



Dipartimento di INGEGNERIA INFORMATICA,
MODELLISTICA, ELETTRONICA E SISTEMISTICA

Corso di Laurea in Ingegneria Informatica

AIOps e Model Context Protocol

Bridge architetturale tra LLM e hypervisor

Relatore:

Prof. Sergio Greco

Candidato:

Michele Arnoni

Matricola: 220121

Correlatore:

Prof. Lucio La Cava

Anno Accademico 2025/2026

Indice

1	Introduzione	1
1.1	Contesto e scenario generale	1
1.1.1	Hypervisor	1
1.1.2	Model Context Protocol (MCP)	1
1.2	Problematiche che si propone di risolvere	1
1.3	Obiettivi della tesi	2
2	Background	3
2.1	Evoluzione delle ITOps e AIOps	3
2.2	Infrastructure as Code (IaC) e API REST	4
2.3	L'interfacciamento degli LLM: dal ReAct al Tool Calling	5
2.4	Limiti degli approcci attuali	5
2.5	Rassegna delle tecnologie alternative	5
3	Analisi dei Requisiti	6
3.1	Requisiti Funzionali	6
3.2	Requisiti Non Funzionali	6
4	Progettazione e scelte architetturali	7
4.1	Architettura	7
4.2	Scelte progettuali	7
5	Sviluppo e implementazione	8
5.1	Funzioni chiave	8
5.2	Sfide tecniche	8
6	Testing e valutazione sperimentale	9
6.1	Soddisfazione dei criteri di test	9
6.2	Metriche di performance e benchmark	9
6.3	Analisi dei risultati	9

7 Conclusioni e sviluppi futuri	10
7.1 Conclusioni oggettive	10
7.2 Sviluppi futuri	10
Bibliografia	11

Sommario

L'integrazione dei *Large Language Model (LLM)* può offrire nuove opportunità per la semplificazione nella gestione e nella manutenzione di infrastrutture server IT.

In particolare, tramite l'emergente protocollo *MCP Server (Model Context Protocol)*, è possibile fornire ad un *LLM* un layer di comunicazione standardizzato con un sistema IT. Questo approccio abilita interazioni basate sul linguaggio umano e portatndo con sé tutta una serie di vantaggi in termini di semplicità e immediatezza operativa per task che tradizionalmente richiederebbero approcci più impegnativi e complessi.

Tuttavia, è importante considerare la natura aleatoria e non deterministica intrinseca di un sistema basato su AI, pertanto questo lavoro di tesi illustra lo sviluppo di un *server MCP* che si relaziona con un *Hypervisor Proxmox*, con annessa valutazione dei limiti e benefici che tale architettura può offrire, ponendo una rigorosa attenzione alla mitigazione dei rischi e alla sicurezza. Attraverso la validazione di questo Proof of Concept, il lavoro dimostra come l'impiego del protocollo *MCP*, unito ad adeguati vincoli architetturali, permetta di governare l'esecuzione di task potenzialmente distruttive in modo controllato e affidabile.

Capitolo 1

Introduzione

1.1 Contesto e scenario generale

1.1.1 Hypervisor

In ambito infrastrutturale IT, una macchina opera grazie all'impiego di sistemi operativi ad hoc.

In tale ambiente si parla infatti di *Hypervisor* [1], ovvero particolari software specializzati nella virtualizzazione multipla di ambienti che ospiteranno i vari servizi.

1.1.2 Model Context Protocol (MCP)

Un qualsiasi *LLM* può interagire con software classici interpretandoli come tools e instaurando con essi una comunicazione simil API, ma molto più specializzata. In questo scenario l'azienda *Anthropic* ha sviluppato un protocollo chiamato *MCP* [2], che permette interazioni strutturate come un sistema client-server, ruoli ricoperti rispettivamente dall'*LLM* e il layer *MCP*.

Attualmente il progetto *MCP* è opensource ed è stato donato da *Anthropic*, per garantirne l'indipendenza, alla *Agentic AI Foundation* [3] [4], un fondo gestito dalla *Linux Foundation*.

1.2 Problematiche che si propone di risolvere

La gestione di un *hypervisor* richiede competenze sistemistiche verticali e l'uso di interfacce GUI o CLI spesso complesse e poco user-friendly. In questo lavoro di tesi si propone di realizzare e valutare un *MCP server* con il ruolo di bridge architetturale tra un qualsiasi *LLM* e un hypervisor aggiungendo così una nuova interfaccia che permetta di interagire con un hypervisor basata sul linguaggio naturale.

1.3 Obiettivi della tesi

L'obiettivo di questo lavoro di tesi è analizzare, implementare e valutare l'efficacia e i limiti di un'architettura *LLM to hypervisor*.

L'architettura non si limita ad interfacciare un *LLM* ad un hypervisor, ma di irrobustire questa interfaccia con scelte architetturali volte a mitigare operazioni potenzialmente distruttive e a mantenere il maggior controllo possibile sulle azioni dell'*LLM*.

Capitolo 2

Background

2.1 Evoluzione delle ITOps e AIOps

Tutte le operazioni volte alla gestione delle infrastrutture IT, quali manutenzione, ottimizzazione e pianificazione delle operazioni strategiche a lungo termine, sono descritte dal termine *ITOps* (*IT Operations*) [5], termine ormai consolidato nel settore, ma che ha subito profonde trasformazioni con l'avvento del cloud computing e la crescente complessità dei sistemi distribuiti.

Tradizionalmente, le attività di ITOps venivano gestite in modo prevalentemente manuale. Gli operatori facevano affidamento su cruscotti di monitoraggio statici e sistemi di alert automatizzati basati su parametri preimpostati.

Con l'esplosione dei dati generati dalle infrastrutture moderne (log, metriche, tracce) e la transizione verso architetture a microservizi, questo approccio si è rivelato difficilmente scalabile e con una inefficienza direttamente proporzionale alle dimensioni del sistema.

La frammentazione dei servizi e l'enorme volume di alert possono rendere molto rumorosa e lenta l'identificazione della causa radice (*root cause analysis*) di un disservizio.

Per rispondere a queste sfide, negli ultimi anni si è assistito all'introduzione delle *AIOps* (*Artificial Intelligence for IT Operations*) [6]. Questo termine, coniato da *Gartner* [7], una delle più importanti e influenti società di ricerca e analisi nel campo IT, descrive come l'impiego di software di monitoring o interazione basato su AI all'interno dei flussi di lavoro delle ITOps possa portare a benefici concreti in termini di incremento dell'immediatezza operativa e di strutturamento di grosse mole di alert.

L'obiettivo principale delle AIOps è aumentare le performance di identificazione e risoluzione di disservizi in tempi più rapidi e di incapsulare automazioni complesse

ed eterogenee in comandi impartiti tramite il linguaggio naturale.

Le soluzioni AIOps operano tipicamente attraverso diverse fasi:

- **Raccolta e aggregazione:** centralizzazione di grandi volumi di dati eterogenei provenienti da molteplici sorgenti dell'infrastruttura.
- **Analisi e rilevamento:** utilizzo di algoritmi di machine learning per l'interpretazione dei dati in tempo reale.
- **Correlazione:** raggruppamento di eventi correlati per ridurre il rumore di fondo ed evitare la duplicazione delle segnalazioni, demoltiplicando gli alert.
- **Risoluzione automatica:** attivazione immediata di workflow automatizzati per mitigare o risolvere il problema senza l'intervento umano diretto o invio di proposte di risoluzione automatizzate con scelta a cura di un umano per le problematiche più critiche.

L'evoluzione verso le AIOps rappresenta quindi un passo fondamentale per garantire la resilienza e l'efficienza dei sistemi IT moderni, ponendo le basi per una gestione infrastrutturale semplificata e intelligente.

2.2 Infrastructure as Code (IaC) e API REST

L'*IaC* (*Infrastructure as Code*) [8] è una pratica ingegneristica che prevede di trattare una infrastruttura IT come un software, con tutti i vantaggi che ne conseguono.

Il workflow di un team IT basato sul paradigma *IaC* si compone delle seguenti fasi:

- **Writing:** scrittura dei file di configurazione architetturale tramite linguaggi tipo *HCL*, *YAML* o *JSON*.
- **Versioning:** i rilasci del codice sorgente vengono strutturati tramite sistemi di controllo delle versioni come *Git*, incorporando anche test di controllo automatici ad ogni nuova versione.
- **Provisioning:** un engine di automazione legge i file di configurazione e ne applica le specifiche in maniera idempotente, ovvero senza duplicare azioni già applicate in precedenza o riassegnare ripetutamente le stesse risorse.
- **Distributing:** si applicano script per la distribuzione dell'infrastruttura in vari ambienti assicurando coerenza e stabilità grazie all'uso dei file sorgente già redatti e testati nella fase di Writing.

È facile intuire come questo approccio, più strutturato e di impronta fortemente ingegneristica, porti concreti vantaggi in termini di controllo del sistema, robustezza, manutenibilità, resilienza all'errore umano, riproducibilità delle configurazioni e scalabilità del sistema.

Ponendo particolare attenzione alla fase di Provisioning e alle tecnologie che la popolano, è possibile imbattersi in strumenti di automazione specifici al sistema su cui operano, come *AWS CloudFormation*, *Azure Resource Manager* o *Google Cloud Deployment Manager*, oppure in strumenti universali come *Terraform* [9]. Questi strumenti si interfacciano con il sistema tramite *API (Application Programming Interface)* permettendo interazioni dirette e programmate ed evitando le interazioni manuali tramite console.

Questo lavoro di tesi vuole evolvere questi strumenti grazie all'AI, rendendoli più flessibili e potenti, sostituendo il tradizionale layer di comunicazione *API* con un *MCP* e permettendo ad un LLM di dialogare con i sistemi IT.

2.3 L'interfacciamento degli LLM: dal ReAct al Tool Calling

Con l'avvento dei primi LLM ogni provider ha provato ad incorporare un modo per poter interfacciare i propri modelli con il mondo esterno e i software tradizionali, che per gli LLM sono interpretati come dei tools. Ma ad un certo punto la società Anthropic ha rilasciato un protocollo open-source volto a standardizzare il modo in cui gli *LLM* si interfacciano con i tool di cui hanno bisogno.

2.4 Limiti degli approcci attuali

2.5 Rassegna delle tecnologie alternative

Capitolo 3

Analisi dei Requisiti

3.1 Requisiti Funzionali

3.2 Requisiti Non Funzionali

Capitolo 4

Progettazione e scelte architettoniche

4.1 Architettura

4.2 Scelte progettuali

Capitolo 5

Sviluppo e implementazione

5.1 Funzioni chiave

5.2 Sfide tecniche

Capitolo 6

Testing e valutazione sperimentale

6.1 Soddisfazione dei criteri di test

6.2 Metriche di performance e benchmark

6.3 Analisi dei risultati

Capitolo 7

Conclusioni e sviluppi futuri

7.1 Conclusioni oggettive

7.2 Sviluppi futuri

Bibliografia

- [1] IBM - Stephanie Susnjara e Ian Smalley. *hypervisor overview*. URL: <https://www.ibm.com/it-it/think/topics/hypervisors>.
- [2] Anthropic. *MCP announcement*. Nov. 2024. URL: <https://www.anthropic.com/news/model-context-protocol>.
- [3] Linux Foundation. *MCP Agentic Ai Foundation*. URL: <https://aaif.io/projects/model-context-protocol>.
- [4] Linux Foundation. *MCP Project Page*. URL: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro>.
- [5] IBM. *ITOps overview*. URL: <https://www.ibm.com/it-it/think/topics/it-operations>.
- [6] IBM. *AIOps overview*. URL: <https://www.ibm.com/it-it/think/topics/aiops>.
- [7] Gartner. *AIOps abstract insights*. URL: <https://www.gartner.com/en/documents/5398763>.
- [8] IBM - Jim Holdsworth e Annie Badman. *Infrastructure as Code overview*. URL: <https://www.ibm.com/it-it/think/topics/infrastructure-as-code>.
- [9] IBM/HashiCorp - Derek Robertson - Matthew Kosinski - Gregg Lindemulder. *Terraform topic*. URL: <https://www.ibm.com/it-it/think/topics/terraform>.